

Nirikshak — Prototype PRD

Status: Draft v1 · prototype phase **Owners:** Shubham Goyal (engineering, integration, console) · Ashutosh Bilthare (extraction, verifier, evaluation) **Hackathon:** AI for Bharat 2026 · Theme 3 · Grand Finale May 16, 2026 · Bangalore **Document purpose:** Single source of truth for what we are building, what we are not building, and how we will know it works on demo day.

0. Reading order

If you are short on time, read §1 (the demo we're building toward), §3 (in-scope vs out-of-scope), §10 (build plan), and §13 (acceptance criteria). Everything else is reference material for when you're working on a specific module.

1. The demo we are building toward

The prototype's success criterion is not "does the system work" — it is "**when a judge watches a five-minute video, do they understand the thesis and believe we have proven it?**"

The thesis we sold in Round 1 is:

┃ AI extracts evidence. Rules decide verdicts. Officers approve.

The prototype demo must make that thesis tangible. The video walks through the **construction-tender scenario from the problem statement** — exactly the four criteria, ten bidders, and 6/3/1 outcome the brief describes — plus one extended scene that shows the disjunctive experience rule (§5.5 of the original proposal) handling a real-shaped CPWD-style criterion. Five segments:

#	Segment	Time	Magic moment
1	Officer uploads tender PDF	0:00–0:45	System extracts the 4 criteria; officer reviews and locks the Criteria Spec. HITL Gate 1 is visible.
2	10 bidder packets ingested in parallel	0:45–1:30	Progress bar; per-bidder evidence cards appear with source-grounded citations (page + bbox). Mixed typed PDFs, scans, and one photograph of a physical certificate.
3	Per-bidder verdicts produced	1:30–2:30	6 Eligible, 3 Not Eligible, 1 Needs Review — exactly the problem-statement outcome. The Not Eligible verdicts show the failed criterion + value + document + page + rule.
4	The disjunctive deep-dive	2:30–3:30	An 11th bidder is evaluated against a CPWD-style "3 at 40% OR 2 at 60% OR 1 at 80%" criterion. All three branches are evaluated and shown. Audit record carries all branches.
5	Audit drill rehearsal	3:30–4:30	A "panellist" asks "why did bidder #6 fail FIN-001?" The system reproduces the verdict from frozen inputs in real time — Criteria Spec hash, bidder document hash, rule that fired, and the cited page region.
6	Three-layer accuracy + scope honesty	4:30–5:00	Closing slide showing OCR/field/verdict accuracy on the Golden Set, with explicit scope limits called out.

This is the build target. Every feature in this PRD justifies its existence by serving one of these segments, validating a thesis claim made in Round 1, or fulfilling an explicit commitment from the Round 1 proposal. Everything else is Phase 2.

2. Goals and success criteria

2.1 Demo-day goals

1. The five segments above land cleanly on a recorded walkthrough and on the live demo at the Bangalore finale.
2. The audit-drill rehearsal works — picked at random by a judge — without rehearsed scripting.
3. The Golden Set accuracy numbers are real, computed on a held-out test set, and we can defend each one.

2.2 Submission deliverables (per hackathon rules)

- Working prototype runnable end-to-end via `docker compose up`

- Five-minute video walkthrough
- Public Git repository (GitHub)
- Idea submission already accepted

2.3 Non-goals (explicit)

We are **not** building any of the following in the prototype phase:

- Production-grade React console (Streamlit is the prototype console)
- Joint Venture aggregation rules (deferred to Phase 2 per Round 1 commitment)
- Forgery / image-tampering detection (deferred per Round 1 commitment)
- Indic language coverage beyond Hindi (English + Hindi end-to-end; other Indic scripts route to Needs Review)
- Live CPPP API integration (no access; we use redacted PDFs that mimic CPPP output)
- Multi-tenant deployment, RBAC, SSO
- True DSC signing with a CCA-licensed certificate (we sign with a self-generated key and explain the production path)
- On-prem LLM serving (we use the Claude API for the prototype; we explain the on-prem swap path with Qwen2.5-VL)

Calling these out explicitly is a feature, not a bug. The demo's closing slide includes them so judges see what is deferred and why.

3. In-scope feature list

The prototype implements a subset of the full architecture described in Round 1. Each feature is tagged with which demo segment it serves.

Feature	Round 1 reference	Demo segment	Owner
Tender PDF ingestion (typed + scanned)	§3.1 Document Understanding	1	Ashutosh
Photograph ingestion (Claude vision)	§3.1	2	Ashutosh
Section classifier (NIT / eligibility / specs / BOQ)	§3.2 Tender Understanding	1	Ashutosh
Criterion miner with mandatory/optional flag	§3.2	1	Ashutosh
Criteria Spec data model (versioned, content-hashed)	§3.2 + §7 Audit	1, 5	Shubham
HITL Gate 1 — Criteria Review console	§6 HITL	1	Shubham
Per-bidder document classifier	§4.1 Bidder Understanding	2	Ashutosh
Per-criterion evidence extractor with Pydantic schemas	§4.2	2	Ashutosh
Source-grounded citations (doc + page + bbox)	§4.2	2, 5	Ashutosh
Verifier pass	§4.2	2	Ashutosh
Per-field confidence scoring	§4.3	2, 3	Ashutosh
Verdict engine with 5 criterion-type rules	§5.1–5.4	3	Shubham
Disjunctive experience rule (the wow factor)	§5.5	4	Shubham
Bidder-level verdict aggregator	§5.4	3	Shubham
HITL Gate 2 — Verdict Review with PDF preview pane	§6 HITL	3	Shubham
Hash-chained audit log	§7.1	5	Shubham
Self-signed PDF report export	§7.2	5	Shubham
Audit drill — replay verdict from frozen inputs	§7.3	5	Shubham
Golden Set with ~10 tenders, ~30 bidder evaluations	§11.2	6	Both
Three-layer accuracy dashboard	§11.2	6	Ashutosh

What is deliberately out, with Round 1 reference:

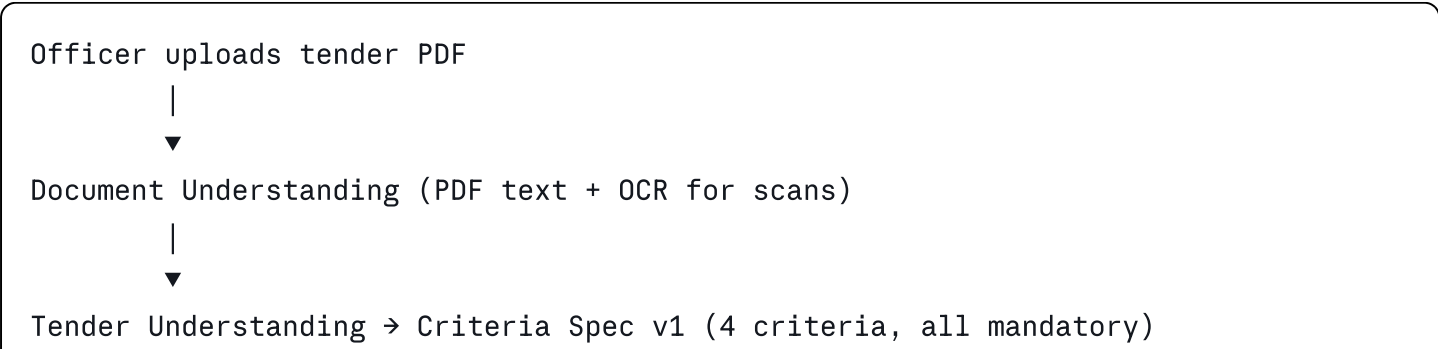
- §4.4 hierarchical financial tables — we will support **simple typed balance sheets**; multi-page hierarchical tables route to Needs Review with a note. Honest scope.
- §6 Edge cases 9-15 (forgery, JV submissions, etc.) — six edge cases ship in the rule engine; the rest are noted in the codebase as TODO with a Phase-2 reference.
- §9 cost-at-scale tiered model usage — we use a single Claude model for the demo and explain the tiered path.

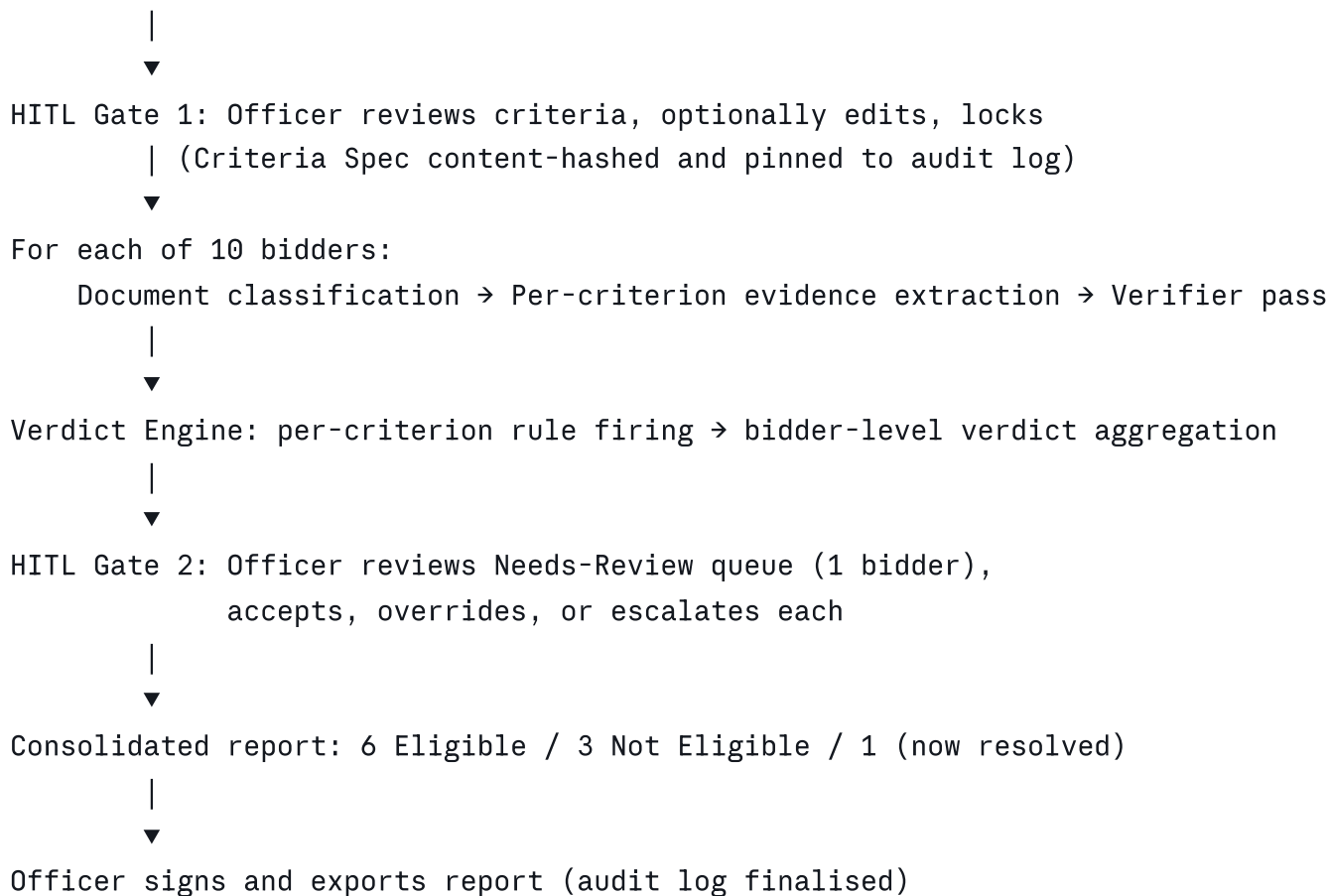
4. User personas and the end-to-end flow

There are three personas the prototype must serve, and one persona for the audit-drill rehearsal:

Persona	Goal	What the system gives them
Procurement officer (primary user)	Evaluate 10 bidders against a tender's eligibility criteria, defensibly	Locked Criteria Spec, queue of per-bidder verdicts with source citations, Needs-Review queue surfaced first, signed report on demand
Tender Evaluation Committee chair (approver)	Sign off on the consolidated report	Read-only view of all verdicts + audit trail; signed PDF export
CAG / CVC auditor (ex-post)	Question any verdict months after the fact	Audit drill: hand the system an audit-log entry hash, get back the full reasoning chain reproduced from frozen inputs
Bidder (out-of-band)	Understand why they were disqualified	Disqualification card in the consolidated report cites failed criterion + value + document + page + rule (sufficient for CPPP representation in 72 hrs per CVC norms)

End-to-end flow on the construction tender





5. Functional requirements — module-by-module

This section is the spec a developer reads before opening their editor. Keep modules small and module boundaries hard.

5.1 Document Understanding (Ashutosh)

Input: A folder of files (PDFs, JPEGs, PNGs).

Output: For each file, a structured `Document` object containing:

- Per-page text (extracted natively where possible, OCR'd otherwise)
- Per-page bounding box index (for citations)
- Document-level content hash
- Routing tag: `native_pdf` | `scanned_pdf` | `photo_certificate`

Implementation:

- Native PDF text via PyMuPDF; if average chars-per-page < 100, treat as scanned.
- Scanned PDFs: PaddleOCR (English + Hindi) per-page, with bbox preserved.
- Photographs: send the page image to Claude Sonnet vision API with a prompt that returns text + structured fields ("Read this certificate. Return issuer, issue date, recipient name, validity").

Success criterion: On the Golden Set, OCR character-error-rate (CER) < 8% on scanned tender pages and < 15% on photographed certificates. We report these numbers honestly in segment 6.

5.2 Tender Understanding (Ashutosh)

Input: A `Document` representing a tender PDF.

Output: A `CriteriaSpec` object — see §6 Data Model.

Pipeline:

1. Section classifier — split tender into segments (NIT, eligibility conditions, technical specifications, BOQ, annexures). Implementation: chunk the document by heading regex first, then a Claude call with structured output for ambiguous chunks.
2. Criterion miner — for each chunk classified as "eligibility conditions", extract criteria as typed objects. Implementation: Claude with a Pydantic schema (see §11). Output includes a `mandatory: bool` flag and a `criterion_type: enum` (one of the six taxonomy types).
3. Officer-readable rendering — natural-language phrasing per criterion plus the structured fields, ready for Gate 1.

Critical correctness check: every extracted criterion must include the source page and a quote span from the original tender. The officer reviews the AI extraction *against the source text*, not in isolation.

5.3 Bidder Evidence Extraction (Ashutosh)

Input: A locked `CriteriaSpec` and a folder of `Document` objects representing one bidder's submission.

Output: For each criterion in the spec, an `EvidenceClaim` object — see §6 Data Model.

Per criterion type, a dedicated extractor:

Type	What it extracts	Schema fields
FinancialThreshold	Turnover / net worth values	<code>amount: Decimal</code> , <code>period: DateRange</code> , <code>source_doc_id</code> , <code>source_page</code> , <code>source_bbox</code> , <code>currency_normalized: bool</code>
ExperienceCount	Completed-work claims	<code>claims: List[CompletedWorkClaim]</code> (each with value, dates, similarity status)
StatutoryRegistration	GST / PAN / EPF / ESI / class+category	<code>registration_number: str</code> , <code>class: str</code> , <code>valid_until: Date</code>
QualityCertification	ISO / AERB / OEM	<code>cert_id: str</code> , <code>version: str</code> , <code>scope: str</code> , <code>expiry: Date</code>
DocumentChecklist	EMD proof / acceptance letter / integrity pact	<code>present: bool</code> , <code>signed: bool</code> , <code>dated: Date</code>
PolicyCompliance	Make-in-India / MSME / debarment	<code>declaration: str</code> , <code>cross_check_status: str</code>

Verifier pass: after extraction, the citation page region is re-fetched from the source document and the extracted value is checked against the actual text. A `Decimal` extraction must round-trip back to the cited region after currency normalisation, with tolerance for OCR digit errors. Failed verification → claim is dropped, criterion routes to Needs Review.

Per-field confidence: each field carries a 0-1 confidence based on (a) OCR confidence in the cited region, (b) LLM self-reported confidence, (c) verifier pass result. Confidence below 0.85 on any mandatory field → criterion routes to Needs Review.

5.4 Verdict Engine (Shubham)

Input: A locked `CriteriaSpec` and a per-bidder `EvidencePool`.

Output: A `BidderVerdict` containing per-criterion verdicts (Eligible / Not Eligible / Needs Manual Review) and an aggregated bidder-level verdict.

Architecture: a small registry pattern. Each criterion type maps to a `Rule` class with a single method `evaluate(criterion, evidence_pool) -> Verdict`. No LLM in this layer — all deterministic Python.

Per-criterion rules (five for the demo plus the disjunctive one):

1. **FinancialThreshold** — windowed turnover/net-worth comparison. Window endpoints computed deterministically from tender's bid-submission date.
2. **StatutoryRegistration** — existence + class/category match + validity-at-submission-date.

3. **QualityCertification** — existence + curated equivalence table (ISO 9001:2008 ↔ :2015) + expiry.
4. **DocumentChecklist** — presence + lightweight content validation (signed, dated, addressed correctly).
5. **PolicyCompliance** — declaration cross-checked against government lists where available.
6. **ExperienceDisjunctive** — see §7 below, this is the centerpiece.

Bidder-level aggregation:

- Any mandatory-criterion `NotEligible` → bidder `NotEligible`.
- Any mandatory-criterion `NeedsReview` (and no `NotEligible`) → bidder `NeedsReview`.
- All mandatory-criteria `Eligible` → bidder `Eligible`.
- Optional-criterion failures do not block — they appear in the report but do not change the bidder's verdict.

5.5 HITL Gates (Shubham)

Gate 1 — Criteria Review:

- Streamlit page rendering each extracted criterion with source quote and page reference.
- Officer can edit any field, add a missing criterion, or remove a falsely-extracted one. All edits versioned with diff against the AI extraction.
- "Lock" button content-hashes the spec, writes to audit log, makes it immutable for downstream evaluation.

Gate 2 — Verdict Review:

- Streamlit page with two panes: left = list of bidders grouped by verdict status; right = selected bidder's verdict with **PDF source pre-highlighted** at the cited bbox.
- For each Needs-Review bidder: officer can Accept-As-Is, Override (with a typed reason), Re-Extract (re-runs evidence extraction with a freeform hint), or Escalate.
- Every officer action writes to the audit log with timestamp and officer identity.

5.6 Audit Log and Report (Shubham)

The audit log is the regulated artefact, not infrastructure. See §8 below for the spec.

The signed PDF report contains:

- Tender summary (title, criteria spec hash, evaluation date)
- Per-bidder verdict cards (bidder name → verdict → for each criterion: rule, evidence value, source citation)
- Audit log summary table

- Self-signed digital signature (production path: CCA-licensed DSC)

5.7 Streamlit Console (Shubham)

Page layout (Streamlit multipage):

Page	Purpose	Demo segment
1_Tender_Library.py	List of tenders, status, recent activity	Demo opens here
2_Criteria_Review.py	Gate 1: criteria spec review and lock	1
3_Bidder_Queue.py	List of bidders for a tender, ingestion progress, per-bidder summary	2
4_Verdict_Review.py	Gate 2: per-bidder verdict with PDF preview pane	3, 4
5_Report_Export.py	Consolidated report preview + signed PDF download	5
6_Audit_Log.py	Audit log viewer with hash chain and replay button	5
7_Eval_Dashboard.py	Golden Set numbers — three-layer accuracy + Needs-Review fraction	6

6. Data model

Implemented as Pydantic models that round-trip to Postgres via SQLAlchemy. The schema is small on purpose — keep it readable.

```
python
```

```
# Core entities
```

```
class Tender(BaseModel):
```

```
    id: UUID
    title: str
    procuring_authority: str # "CRPF Zone Bangalore"
    bid_submission_date: date
    estimated_value: Decimal
    source_pdf_id: UUID
```

```
class CriteriaSpec(BaseModel):
```

```
    id: UUID
    tender_id: UUID
    version: int
    criteria: List[Criterion]
    content_hash: str # SHA-256, written to audit log on lock
    locked_at: datetime | None
    locked_by: str | None
```

```
class Criterion(BaseModel):
```

```
    id: str # e.g., "FIN-001"
    type: CriterionType # enum
    description: str
    mandatory: bool
    parameters: dict # type-specific (threshold, window, etc.)
    source_page: int
    source_quote: str
```

```
class Bidder(BaseModel):
```

```
    id: UUID
    tender_id: UUID
    name: str
    submission_date: datetime
    document_ids: List[UUID]
```

```
class Document(BaseModel):
```

```
    id: UUID
    bidder_id: UUID | None # None if it's a tender doc
    tender_id: UUID | None
    filename: str
    content_hash: str
    routing_tag: Literal["native_pdf", "scanned_pdf", "photo_certificate"]
    pages: List[Page]
```

```
class Page(BaseModel):
```

```
    page_number: int
```

```

text: str
bboxes: List[BBox] # word-level for citation

class EvidenceClaim(BaseModel):
    bidder_id: UUID
    criterion_id: str
    extracted_value: Any # type-specific
    source_doc_id: UUID
    source_page: int
    source_bbox: BBox
    confidence: float
    verifier_passed: bool

class Verdict(BaseModel):
    bidder_id: UUID
    criterion_id: str
    state: Literal["Eligible", "NotEligible", "NeedsReview"]
    evidence_used: List[EvidenceClaim]
    rule_fired: str # e.g., "FinancialThreshold.WindowedTurnover"
    reason_template: str # filled-in natural language
    officer_action: OfficerAction | None # set when Gate 2 closes

class BidderVerdict(BaseModel):
    bidder_id: UUID
    aggregate_state: Literal["Eligible", "NotEligible", "NeedsReview"]
    per_criterion: List[Verdict]
    finalized_at: datetime | None

class AuditLogEntry(BaseModel):
    sequence: int
    timestamp: datetime
    actor: str # "system" or officer email
    action_type: str # "ingest" | "extract" | "rule_fire" | "officer_edit" | ...
    payload_hash: str # hash of the action's input/output
    previous_hash: str # hash chain
    entry_hash: str # hash of (sequence + timestamp + payload_hash + previous_hash)

```

7. The disjunctive experience rule — the centerpiece

This is the segment-4 wow factor. We treat it with disproportionate care because it is the single most defensible argument that Nirikshak does something a generic extractor cannot.

The criterion (verbatim from CPWD-style tenders we cited in Round 1 §5.5):

"Three similar completed works each $\geq 40\%$ of estimated cost; OR two at $\geq 60\%$; OR one at $\geq 80\%$, in the last seven years ending the last day of the month previous to the one in which the bid is invited."

7.1 What the prototype must show

For one seeded bidder (call them "Bidder K"), with a messy real-shaped experience portfolio (5 completion certificates, mixed values, mixed dates, one borderline-similar work), the system must:

1. Extract each completion claim as a typed `CompletedWorkClaim` with cited completion certificate.
2. Compute the temporal window endpoints deterministically from the tender's bid-invitation date.
3. Filter claims to the window.
4. For each filtered claim, classify similarity to the tender scope (a small fine-tuned classifier; for the prototype, a Claude call with a strict prompt and side-by-side text comparison).
5. Evaluate all three branches independently:
 - Branch A: ≥ 3 claims at $\geq 40\%$ of estimated cost — pass/fail
 - Branch B: ≥ 2 claims at $\geq 60\%$ — pass/fail
 - Branch C: ≥ 1 claim at $\geq 80\%$ — pass/fail
6. Criterion passes if any branch passes.
7. **The audit record carries all three branch evaluations, not just the winning one.**

7.2 The verdict explanation we render

Criterion: EXP-001 (mandatory)

"Three similar completed works each $\geq 40\%$ of estimated cost; OR two at $\geq 60\%$; OR one at $\geq 80\%$,
in the last seven years ending [computed window end]."

Window: 2018-04-30 $\rightarrow$ 2025-04-30 (computed from bid-invitation date)

Estimated cost: ₹50 crore

Thresholds: 40% = ₹20 cr, 60% = ₹30 cr, 80% = ₹40 cr

Bidder K's claims after window filter and similarity classification:

Claim 1: ₹22.5 cr — similar — completion cert p.4 [HIGHLIGHT]

Claim 2: ₹18.0 cr — similar — completion cert p.7 [HIGHLIGHT] $\leftarrow$ below 40%,
drops out

Claim 3: ₹35.0 cr — similar — completion cert p.11 [HIGHLIGHT]

Claim 4: ₹42.0 cr — similar — completion cert p.14 [HIGHLIGHT]

Claim 5: ₹28.0 cr — borderline-similar (routed to Needs Review)

Branch A (≥ 3 at $\geq 40\%$): claims meeting threshold = {1, 3, 4} $\rightarrow$ 3 claims $\rightarrow$ PASS
Branch B (≥ 2 at $\geq 60\%$): claims meeting threshold = {3, 4} $\rightarrow$ 2 claims $\rightarrow$ PASS
Branch C (≥ 1 at $\geq 80\%$): claims meeting threshold = {4} $\rightarrow$ 1 claim $\rightarrow$ PASS

Verdict: ELIGIBLE (all three branches pass; criterion passes if any branch passes)

This is what segment 4 of the demo shows on screen.

8. Audit log spec

The audit log is the regulated artefact, not a side feature. It must satisfy three requirements:

1. **Tamper-evident** — any modification to a past entry breaks the chain.
2. **Reproducible** — given an entry hash, we can reconstruct the exact reasoning that produced it.
3. **Query-able from a procurement officer's perspective** — "show me everything that happened to bidder K's verdict on FIN-001."

8.1 Hash chain

Each entry contains:

- Monotonic sequence number
- Server timestamp (UTC)
- Actor (system module or officer email)
- Action type
- Payload hash (SHA-256 of JSON-serialized input/output of the action)
- Previous entry hash
- Entry hash = SHA-256 of (sequence + timestamp + actor + action_type + payload_hash + previous_hash)

Entries are append-only. The DB schema enforces no-update-no-delete via Postgres RULE.

8.2 Action types we log

Action	When	Payload
tender_ingested	Tender PDF first uploaded	tender_id, document content hash
criteria_extracted	After tender understanding	criteria_spec v1 content hash

Action	When	Payload
<code>criteria_locked</code>	Officer commits Gate 1	locked criteria_spec hash, officer email
<code>bidder_ingested</code>	Each bidder packet uploaded	bidder_id, list of doc content hashes
<code>evidence_extracted</code>	Per criterion per bidder	criterion_id, evidence object hash
<code>rule_fired</code>	Verdict engine	criterion_id, rule name, verdict state
<code>bidder_verdict</code>	Aggregator	bidder_id, aggregate verdict
<code>officer_review</code>	Gate 2 action	bidder_id, action (accept/override/re-extract/escalate), reason
<code>report_finalized</code>	Officer signs off	report content hash, signature

8.3 Replay (the audit drill)

Given an entry hash, the replay function:

1. Fetches the entry and follows the chain backward to reconstruct the full action history for that bidder + criterion.
2. Re-loads the locked Criteria Spec (by hash).
3. Re-loads the bidder's documents (by content hash) — they live in immutable object storage.
4. Re-runs the evidence extractor and verdict engine.
5. Compares the recomputed verdict to the historical verdict. If they differ, emits a divergence report (this should never happen in production but the divergence report is the safety net).

For the demo, segment 5 invokes replay live on bidder #6's FIN-001 verdict and shows the steps in real time.

9. Golden Set and three-layer accuracy

9.1 Composition

We curate 8–10 representative tenders sourced from publicly indexed CPPP archives (redacted where necessary), spanning:

- 3× construction (CPWD/CRPF style — most of our criterion patterns)
- 2× vehicles / equipment procurement
- 2× security services

- 1× IT services or office supplies (different criterion shape)
- 1× edge case: a tender with an unusually phrased criterion

For each tender, we hand-curate ground-truth evaluations on **3–5 mock bidders** (synthetic but realistic), giving us approximately 30–40 bidder-criterion pairs as our test set.

This is smaller than the 30 tenders we mentioned in Round 1. We make this scope reduction explicit in the demo's segment 6 — better to have 8 tenders fully labelled and analysed than 30 half-baked.

9.2 The three layers

We deliberately do not report a single end-to-end accuracy number. Instead, three independent layers — because end-to-end claims usually mask easier sub-tasks (a point made in Round 1 §11.2):

Layer	What we measure	Method
OCR character-level	Character error rate on extracted text	CER on a sample of pages, separately for typed PDFs / scanned PDFs / photographs
Field-level extraction	Did the extractor produce the correct typed value?	Per-field comparison against ground truth, with currency / date / entity normalisation
Verdict-level agreement	Does Nirikshak's verdict match the human evaluation?	Per-criterion-per-bidder cohen's kappa-style agreement

Plus a fourth metric that tells the system's honest story:

Metric	Why it matters
Needs-Review fraction	If too low → system is over-confident; if too high → system is dodging hard cases. Both are bad. We aim for the rate where disagreements concentrate.

9.3 Demo dashboard

Segment 6 closes with a single screen showing all four numbers, the test set size, and the explicit caveats:

Numbers below are computed on a held-out test set of 8 tenders × ~32 bidder evaluations. Production calibration would require ~200 real evaluations to lock thresholds.

10. Build plan and timeline

Two builders, calendar days. The plan assumes Shubham and Ashutosh both work

approximately full-time on this in the lead-up to the prototype submission, with finals-day polish at the Bangalore venue on May 16.

10.1 Day-by-day plan

Day	Shubham	Ashutosh	End-of-day deliverable
1	Repo, Docker compose, Postgres + pgvector, Pydantic schemas, hash-chain skeleton	Claude API client wrapper, PyMuPDF ingestion, page render for highlighting	<code>docker compose up</code> works, can ingest one PDF
2	Audit log writes; Streamlit shell; Tender Library page	PaddleOCR integration, document routing tag, OCR confidence	Tender uploaded, pages indexed, audit log shows ingestion
3	Criteria Spec data model + lock flow; Gate 1 page	Section classifier + criterion miner with Pydantic structured output	Tender → criteria spec end-to-end; officer can lock Gate 1
4	Bidder data model; Bidder Queue page; document classifier shell	Per-criterion evidence extractor for FinancialThreshold and StatutoryRegistration types	Two bidders ingested; financial + GST evidence extracted with citations
5	Verdict engine framework + FinancialThreshold + StatutoryRegistration rules	Evidence extractors for QualityCertification + DocumentChecklist + PolicyCompliance	Five-rule verdict engine works on one bidder
6	The disjunctive rule (§7) + bidder-level aggregator	ExperienceCount evidence extractor with similarity classification	Disjunctive demo scene works on Bidder K (segment 4)
7	Gate 2 page with PDF preview + bbox highlighting	Verifier pass + per-field confidence scoring	All 10 demo bidders run end-to-end on construction tender
8	Signed PDF report + audit log viewer page	Golden Set curation (4 of ~8 tenders)	Report exports correctly; first eval numbers available
9	Audit drill (replay function) + Eval dashboard page	Golden Set curation finished + shadow-eval run + accuracy numbers	Three-layer accuracy on dashboard; replay works

Day	Shubham	Ashutosh	End-of-day deliverable
10	Demo polish, README, architecture diagram	Prepare seed data scripts, recorded demo walkthrough	Submission-ready: GitHub repo + 5-minute video

10.2 Risk-adjusted milestones

Hard checkpoints — if we slip, we cut scope, not quality:

- **End of Day 4:** Two bidders ingested, evidence extracted, audit log writing. *If we slip:* drop one criterion type (PolicyCompliance) entirely.
- **End of Day 6:** Disjunctive rule works on Bidder K. *If we slip:* simplify Bidder K's portfolio to 3 claims so the Branch B/C math is trivial; the demo scene still lands.
- **End of Day 8:** All 10 bidders run end-to-end. *If we slip:* drop to 6 bidders (still preserves the 6/3/1 narrative — 4 Eligible + 1 Not Eligible + 1 Needs Review). We never sacrifice the disjunctive scene.
- **End of Day 9:** Replay works. *If we slip:* hard-code the replay narrative for the demo recording but keep the live system honest. (Document this as a known limitation.)

10.3 Onsite at Bangalore (May 16)

The onsite is for refinement, not new features. Plan:

- Morning: dry-run the demo on the venue's network; verify Claude API access; pre-warm caches.
- Pre-judging: 30-minute tech check; have a 60-second elevator pitch ready in case the panel asks for a one-liner.
- Mentor sessions: prioritise feedback that strengthens the audit-drill narrative and the Golden Set numbers — those are what skeptical IAS/CAG-aligned judges will challenge.

11. Repository structure

```
nirikshak/
├─ README.md
├─ docker-compose.yml
├─ pyproject.toml
├─ .env.example
├─
├─ nirikshak/                # Main Python package
│   ├── __init__.py
│   └─ core/
```

```

├─ schemas.py                # All Pydantic models (§6)
├─ hashing.py                # content hashing utilities
├─ config.py
├─ db.py                     # SQLAlchemy session factory
├─
├─ ingestion/
│   ├─ pdf_text.py           # PyMuPDF wrapper
│   ├─ ocr.py                # PaddleOCR wrapper
│   ├─ vision.py             # Claude vision for photos
│   ├─ classify.py            # routing tag detection
│   └─ page_render.py        # render page with bbox highlight
├─
├─ tender/
│   ├─ section_classifier.py
│   ├─ criterion_miner.py
│   └─ criteria_spec.py      # spec construction + locking
├─
├─ bidder/
│   ├─ doc_classifier.py
│   ├─ extractors/
│   │   ├─ base.py
│   │   ├─ financial_threshold.py
│   │   ├─ experience_count.py    # produces CompletedWorkClaim list
│   │   ├─ statutory_registration.py
│   │   ├─ quality_certification.py
│   │   └─ document_checklist.py
│   └─ policy_compliance.py
├─ verifier.py               # cited-region re-check
├─ confidence.py
├─
├─ verdict/
│   ├─ engine.py             # registry + dispatch
│   ├─ rules/
│   │   ├─ financial_threshold.py
│   │   ├─ experience_disjunctive.py    # ★ centerpiece
│   │   ├─ statutory_registration.py
│   │   ├─ quality_certification.py
│   │   └─ document_checklist.py
│   └─ policy_compliance.py
├─ aggregator.py
├─
├─ audit/
│   ├─ chain.py              # hash-chain implementation
│   ├─ signer.py             # self-signed PDF (DSC stub)
│   └─ replay.py             # the audit drill
├─
├─ llm/

```

```

| | |└─ client.py # Claude wrapper with retries
| | |└─ instructor_helpers.py
| | |└─ prompts/ # one prompt per file
| | |   └─ section_classifier.md
| | |   └─ criterion_miner.md
| | |   └─ financial_extractor.md
| | |   └─ experience_extractor.md
| | |   └─ similarity_classifier.md
| | |   └─ ...
|
|└─ eval/
| |└─ golden_set.py # loader for hand-curated dataset
| |└─ runner.py # shadow-eval runner
| |└─ metrics.py # OCR / field / verdict metrics
|
|└─ api/
| |└─ main.py # FastAPI endpoints
|
|└─ console/
| |└─ streamlit_app.py
| |└─ pages/
| |   └─ 1_Tender_Library.py
| |   └─ 2_Criteria_Review.py
| |   └─ 3_Bidder_Queue.py
| |   └─ 4_Verdict_Review.py
| |   └─ 5_Report_Export.py
| |   └─ 6_Audit_Log.py
| |   └─ 7_Eval_Dashboard.py
|
|└─ golden_set/ # checked-in test data
| |└─ tenders/ # 8 redacted PDFs
| |└─ bidders/ # mock submissions
| |└─ ground_truth.yaml
|
|└─ seed/ # demo seed data
| |└─ construction_tender.pdf # the demo's primary tender
| |└─ 10_bidders/ # the demo's bidder set
| |└─ bidder_K/ # disjunctive-rule scenario
|
|└─ tests/
| |└─ unit/
| | |└─ test_disjunctive.py # exhaustive on the wow-factor rule
| | |└─ test_audit_chain.py
| | |└─ ...
| |└─ integration/
| | |└─ test_e2e_construction.py # ingest → verdict → report

```

└ docs/	
├ PRD.md	# this document
├ ARCHITECTURE.md	# diagram + module map
├ DEMO_SCRIPT.md	# the 5-minute video script
└ HONEST_SCOPE.md	# what's out and why

12. The 5-minute video script

Owner: both. Recorded on day 10. Script structure (rough timing in seconds):

00-15 — Opening. Black slide: "AI extracts evidence. Rules decide verdicts. Officers approve."
Voice-over: 1-2 sentences on the problem.

15-60 — Segment 1: Officer uploads the construction tender. Click → criteria appear. Voice-over highlights the mandatory/optional flag and the source-quote citations. Officer locks Gate 1. The lock event is shown in the audit log on a side pane.

60-105 — Segment 2: Bidder Queue page. Click "ingest 10 bidders." Progress bar. Per-bidder evidence cards appear with thumbnails of the cited document pages. Voice-over points out the heterogeneous formats (typed, scanned, one photograph).

105-195 — Segment 3: Verdict Review page. The Eligible/Not Eligible/Needs Review tabs show 6/3/1 — match the problem statement. Click into one Not Eligible bidder; show the verdict card with rule name, evidence value, and pre-highlighted source PDF region. Resolve the Needs-Review case.

195-270 — Segment 4: The disjunctive deep dive. Open Bidder K. The verdict view shows the criterion in full, the computed window, the threshold values, the five claims with similarity classification, and **all three branches evaluated**. Voice-over: "This is the rule pattern manual evaluators struggle with most. We resisted the temptation to let the LLM judge — every step you see is deterministic Python."

270-330 — Segment 5: The audit drill. Cut to Audit Log page. Voice-over: "Months from now, an auditor asks: why did Bidder #6 fail FIN-001?" Click the replay button on Bidder #6's FIN-001 entry. The system reconstructs the verdict from the locked Criteria Spec, the bidder's content-hashed documents, and the rule that fired. The recomputed verdict matches the historical one.

330-360 — Segment 6: Eval dashboard with three-layer accuracy + Needs-Review fraction. Closing slide with explicit honest scope: "We have built around the trustworthiness constraint, not the accuracy headline number. The audit log, not the model accuracy, is the regulated artefact this system stakes its credibility on."

13. Acceptance criteria (the prototype is "done" when...)

Every criterion below is binary. We are done when every box is checked.

Functional:

- ☐ `docker compose up` brings up the system on a fresh laptop in under 5 minutes
- ☐ Tender PDF upload → Criteria Spec on screen within 60 seconds
- ☐ All 10 demo bidders ingest and produce verdicts within 5 minutes
- ☐ Verdict cards show correct citation page + bbox highlight in the PDF preview
- ☐ Disjunctive rule on Bidder K produces all three branch evaluations
- ☐ Replay function on any audit-log entry reproduces the same verdict as recorded
- ☐ Signed PDF report exports and contains all 10 bidder verdicts
- ☐ Golden Set runs end-to-end and produces all four metrics on the dashboard

Non-functional:

- ☐ Audit log entries are append-only (verified by attempted UPDATE failing)
- ☐ Hash-chain verification function passes on the entire log after a demo run
- ☐ No bidder is silently disqualified — every NotEligible verdict carries the cited evidence
- ☐ No claim is in any verdict that does not appear in the cited document (verifier pass enforces this)

Submission deliverables:

- ☐ Public GitHub repository with README, architecture diagram, demo data, instructions
- ☐ 5-minute video walkthrough recorded and uploaded
- ☐ HONEST_SCOPE.md committed at root, listing what's deferred and why

14. Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Claude API rate limits hit during demo	Medium	High	Cache all extraction results in Postgres after first run; the "live" demo replays from cache
OCR fails on a key Hindi page in a demo bidder	Medium	Medium	Curate demo bidders to use English text; have Hindi-only as a stretch case
Disjunctive rule has an off-by-one in branch evaluation	Medium	High (it's the wow factor)	Exhaustive unit tests on Bidder K's exact data; freeze that bidder's data on Day 7
Replay doesn't match historical verdict (non-determinism)	Medium	High	Pin LLM model versions; freeze prompts; cache extraction outputs; replay re-loads from cache, not the LLM
PDF preview / bbox highlighting flaky	Medium	Medium	Pre-render highlighted page images on first verdict computation; serve from disk in the UI

Risk	Likelihood	Impact	Mitigation
Golden Set numbers are embarrassingly low	Low	High	Choose the 8 tenders carefully — straightforward criterion patterns dominate; flag harder cases as Needs-Review
One of us gets pulled into work emergencies	Medium	High	Ship Day 6 milestone (end-to-end on construction tender) as a hard fork point — even if the eval and polish slip, the core demo runs
Onsite venue's network blocks Claude API	Low	High	Cache everything before flying to Bangalore; demo can run fully from cache

15. Open questions (resolve before Day 1)

- LLM choice for production claim.** We use Claude Sonnet for the prototype. In the demo, do we say "the production deployment uses Qwen2.5-VL on-prem" (per Round 1) or do we say "Claude Sonnet end-to-end with on-prem swap as a future step"? Recommend the former — it's what we promised.
- Bidder K's data.** Do we generate it synthetically (cleaner) or curate a real-shaped redacted bidder packet (more credible)? Recommend a hybrid — synthetic numbers, real-shaped completion certificate templates.
- Officer identity in demo.** Stub user "officer1@crpf.gov.in" or a dropdown of three named officers? Recommend single stubbed officer for simplicity; surface this as a "single-tenant prototype" honest scope item.
- Repo visibility.** Public from Day 1 or private until submission? Recommend private until Day 8 then public — protects against scrappers, gives time to clean up commit history.

Appendix A — Worked example: the construction tender

(Full version moves into the demo's seed data; here's the spec)

Tender: Construction of operational vehicle parking and security shed — CRPF Zone Bangalore.

Estimated cost: ₹50,00,00,000 (₹50 crore).

Bid invitation date: 2025-04-30.

Eligibility criteria (extracted by tender understanding):

ID	Type	Description	Mandatory
FIN-001	FinancialThreshold	Average annual turnover ≥ ₹5 cr in last 3 FYs	Yes
EXP-001	ExperienceCount	At least 3 similar projects completed in last 5 years	Yes
REG-001	StatutoryRegistration	Valid GST registration	Yes
QUA-001	QualityCertification	ISO 9001 (any version 2008 or 2015)	Yes

Bidders 1-10: mix of typed and scanned packets, designed to produce 6 Eligible / 3 NotEligible / 1 NeedsReview per the problem statement.

Bidder K (the wow factor): evaluated against an additional EXP-002 disjunctive criterion as described in §7.

Appendix B — Out-of-scope acknowledgement (lifted into HONEST_SCOPE.md)

Round 1 commitments we **deliver** in the prototype:

- The thesis architecture (LLM extracts, rules decide, officers approve)
- HITL Gates 1 and 2
- Source-grounded citations with verifier pass
- Hash-chained audit log + replay
- Six criterion types in the rule engine including the disjunctive one
- Three-layer accuracy reporting
- Honest Needs-Review fraction

Round 1 commitments we **defer to Phase 2** (with explanation):

- JV aggregation rules
- Forgery / image-tampering detection
- Indic language coverage beyond Hindi (other Indic scripts route to Needs Review)
- Hierarchical multi-page balance-sheet parsing (we handle simple typed balance sheets)
- Production-grade React console (Streamlit for prototype)
- True DSC signing with CCA-licensed certificate (self-signed for prototype)
- On-prem LLM serving (Claude API for prototype)

End of PRD v1. Edits welcome — this document lives in the repo and gets updated as the build progresses.

